
Sistemas Operacionais

Interação entre Tarefas - Parte II

Rogério Pereira Junior

Instituto Federal de Santa Catarina
campus São José
rogeriopereirajunior@gmail.com

19 de maio de 2026



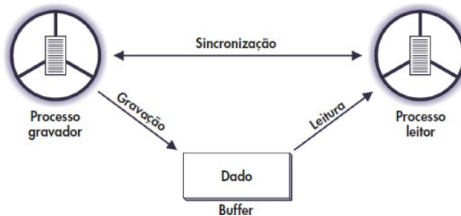
- Os processos são isolados pelo Sistema Operacional para segurança.
- **IPC (Inter-Process Communication):** Conjunto de mecanismos que permitem que processos troquem dados.
- **Principais mecanismos:**
 - 1 Pipes (Tubulações)
 - 2 Filas de Mensagens (Message Queues)
 - 3 Memória Compartilhada (Shared Memory)



Coordenação entre tarefas



- Muitas implementações de sistemas complexos são estruturadas como várias tarefas interdependentes
 - Cooperam entre si
 - Necessidade de comunicação para conseguir a cooperação
- Problemas podem ocorrer nesse processo de comunicações
- Veremos esses problemas e algumas possíveis soluções (simples em primeiro momento).



Quais problemas podem ocorrer quando 2 ou mais tarefas acessam de maneira concorrente os mesmos recursos?



Concorrência

- Capacidade de lidar com múltiplas tarefas ao mesmo tempo.
- Não significa execução simultânea, mas alternância entre tarefas.
- Exemplo: multitarefa em um processador de único núcleo.

Paralelismo

- Execução real e simultânea de múltiplas tarefas.
- Requer múltiplos núcleos ou processadores.
- Exemplo: processamento paralelo em um computador com vários núcleos.



Aspecto	Concorrência	Paralelismo
Definição	Alternância entre tarefas	Execução simultânea
Foco	Coordenação	Performance
Hardware	Pode ocorrer em 1 núcleo	Precisa de múltiplos núcleos
Exemplo	Multitarefa em PC comum	Supercomputadores, CPUs multi-core



O problema da concorrência

Quando duas ou mais tarefas acessam simultaneamente um recurso compartilhado, podem ocorrer problemas de consistência dos dados ou do estado do recurso acessado.

Exemplo de Condição de Corrida



Exemplo: Depósito Bancário

Considere uma função simples de depósito:

Código C

```
void depositar (long conta, long valor) {  
    (*saldo) += valor;  
}
```

- Em nível de montagem (Assembly), essa linha se torna várias instruções: carregar memória, somar no registrador e salvar de volta.
- **Analogia:** Duas pessoas tentando escrever em um post-it ao mesmo tempo. Se ambas leem "0", e uma soma 50 e a outra 1000, o resultado final dependerá de quem "colar" o post-it por último.



Exemplo: Depósito Bancário

- gcc -Wall -c -O0 depositar.c
- objdump -no-show-raw-insn -d depositar.o

```
0000000000000000 <depositar>:
    ; carrega o conteúdo da memória apontada por "saldo" em EDX
    mov     -0x8(%rbp),%rax      ; saldo → rax (endereço do saldo)
    mov     (%rax),%edx         ; mem[rax] → edx

    ; carrega o conteúdo de "valor" no registrador EAX
    mov     -0xc(%rbp),%eax     ; valor → eax

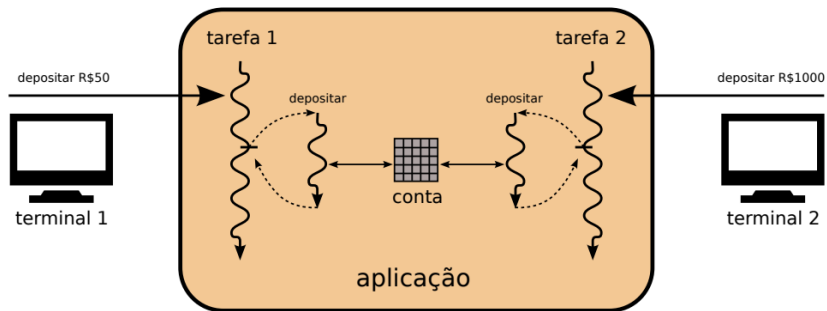
    ; soma EAX ao valor em EDX
    add     %eax,%edx           ; eax + edx → edx

    ; escreve o resultado em EDX na memória apontada por "saldo"
    mov     -0x8(%rbp),%rax     ; saldo → rax
    mov     %edx,(%rax)         ; edx → mem[rax]
```

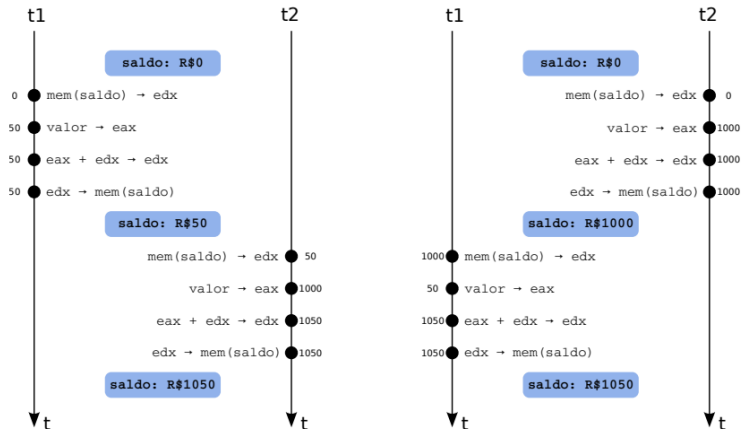
- As operações aritméticas ocorrem nos registradores
 - O valor da variável saldo (que está na memória) é copiado para um registrador da CPU (um espaço interno ultrarrápido).
 - a soma **temp** += **valor** não é feita diretamente na memória, mas dentro do registrador. O processador pega o valor de **temp** e **valor**, coloca-os em registradores e executa a soma ali.



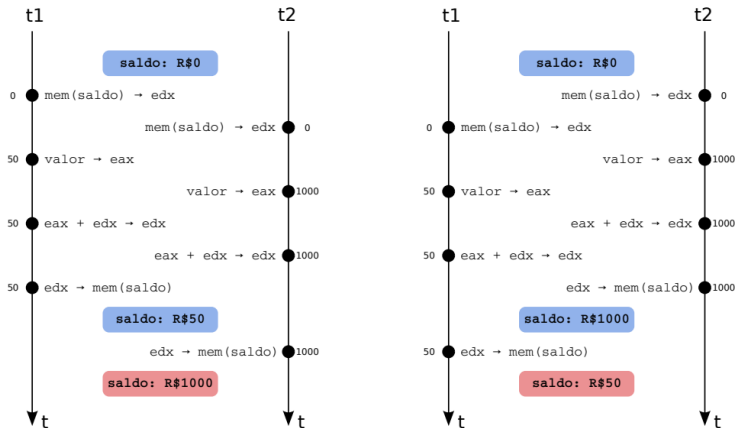
Uma aplicação concorrente



Execuções independentes



Execuções concorrentes



Uma aplicação concorrente

```
#include <stdio.h>
#include <pthread.h>

int saldo = 0; // recurso compartilhado

void* depositar(void* arg) {
    int valor = *(int*)arg;
    int temp = saldo;    // lê o saldo
    temp += valor;       // soma o depósito
    saldo = temp;        // escreve o novo saldo
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int v1 = 50, v2 = 1000;

    pthread_create(&t1, NULL, depositar, &v1);
    pthread_create(&t2, NULL, depositar, &v2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Saldo final: %d\n", saldo);
    return 0;
}
```



Condição de Disputa (Race Condition)

- O que vimos no exemplo da conta se chama **condição de corrida/disputa**.
- Erros gerados por acessos concorrentes a dados
- O resultado final depende do entrelaçamento (timing) das tarefas.
- Se as operações se intercalam, um depósito pode ser "perdido".
- Envolvem ao menos uma operação de escrita
- Podem ocorrer em qualquer sistema concorrente

Quando acontece?

Quando duas ou mais threads ou processos acessam e manipulam um recurso compartilhado (como uma variável na memória) ao mesmo tempo, e o resultado final depende da ordem em que essas operações ocorrem.

Consequência = resultados imprevisíveis ou incorretos.



Condição de Disputa (Race Condition)

Condições de disputa são erros dinâmicos:

- Não aparecem no código fonte
- Só se manifestam durante a execução
- São difíceis de detectar
- Podem ocorrer raramente ou mesmo nunca
- Sua depuração pode ser muito complexa

Definição Formal

São erros que não aparecem no código-fonte e só se manifestam durante a execução, sendo raros e difíceis de depurar.

É melhor prevenir condições de disputa que consertá-las.

Uma condição de disputa é um fantasma que pode assombrar seu código por anos sem nunca ser visto.



Formalizam quando duas tarefas (t_1 e t_2) podem rodar em paralelo sem riscos:

As Regras ($t_1 || t_2$)

- 1 $R(t_1) \cap W(t_2) = \emptyset$ (t_1 não lê o que t_2 escreve)
- 2 $R(t_2) \cap W(t_1) = \emptyset$ (t_2 não lê o que t_1 escreve)
- 3 $W(t_1) \cap W(t_2) = \emptyset$ (ambas não escrevem no mesmo lugar)

Dica de Ouro

Leituras simultâneas não geram disputa. O problema nasce quando há pelo menos uma **escrita**.



- Trechos de código que manipulam dados compartilhados.
- As seções críticas das tarefas t1 e t2 são:

```
(*saldo) += valor;
```

- se várias threads entram na seção crítica ao mesmo tempo, há risco de inconsistência.

```
int temp = saldo;    // lê o saldo
temp += valor;       // soma o depósito
saldo = temp;        // escreve o novo saldo
```

- Se duas threads executarem esse trecho simultaneamente, pode ocorrer uma condição de disputa e o resultado final será incorreto.

Definição

é o trecho do código onde ocorre o acesso a um recurso compartilhado que não pode ser usado por mais de uma thread ao mesmo tempo sem causar problemas.



- Seção crítica é como uma ponte estreita de mão única.
- Só cabe um carro por vez atravessando.
- Se dois carros tentarem entrar juntos, vão bater ou ficar travados.
- A ponte representa o trecho de código que acessa o recurso compartilhado.



Solução para ponte: Colocar um mecanismo de controle de acesso.

Solução para SO: Usar mecanismos de exclusão mútua.



Exclusão Mútua

Propriedade de garantir que apenas uma tarefa esteja em sua seção crítica por vez.

Analogia do Banheiro

A seção crítica é como um banheiro de uso individual: para entrar, você precisa da chave (enter). Enquanto estiver lá dentro, ninguém mais entra. Ao sair, você devolve a chave (leave).

Foco

Impedir o entrelaçamento de seções críticas, de modo que apenas uma tarefa esteja na seção crítica a cada instante.



Exclusão Mútua: A Regra de Ouro

- **O que é:** Uma propriedade que garante que, se uma tarefa está executando em sua seção crítica, nenhuma outra tarefa pode entrar na mesma seção crítica.

Analogia: A Cabine Telefônica

Imagine uma cabine telefônica antiga:

- 1 Apenas uma pessoa cabe lá dentro por vez.
- 2 Enquanto alguém fala ao telefone (Seção Crítica), os outros esperam do lado de fora.
- 3 Somente quando a pessoa sai (Liberação), a próxima pode entrar.

Primitivas de Controle

Para funcionar, o programador deve delimitar o código com:

- **Enter(recurso):** Solicita permissão (bloqueia se ocupado).
- **Leave(recurso):** Avisa que terminou e libera o acesso.



- Para evitar condições de disputa, o acesso às variáveis compartilhadas deve ser protegido.
- O código da operação de depósito pode ser reescrito assim:

```
void depositar (long conta, long *saldo, long valor){  
    enter (conta) ; // entra na seção crítica "conta"  
    (*saldo) += valor ; // usa as variáveis compartilhadas  
    leave (conta) ; // sai da seção crítica  
}
```

- A variável **conta** identifica a seção crítica.
- As funções **enter()** e **leave()** garantem a exclusão mútua:
 - Apenas uma thread pode acessar a seção crítica por vez.
 - Evita inconsistências no recurso compartilhado.



Critérios para Exclusão Mútua

Para que um mecanismo de exclusão mútua seja considerado correto e robusto, ele deve atender a quatro requisitos fundamentais:

Os 4 Pilares

- Exclusão Mútua** Apenas uma tarefa por vez na seção crítica.
- Espera Limitada** Nenhuma tarefa deve esperar para sempre (fim da inanição).
- Independência** Tarefas fora da seção crítica não podem impedir outras de entrar.
- Fatores Físicos** A solução deve ser lógica (não depender da velocidade da CPU ou número de núcleos).

Uma solução de exclusão mútua que depende da 'sorte' do escalonador não é uma solução, é uma bomba relógio.



- Vamos ver um exemplo que temos o problema de condição de corrida
- Vamos identificar a seção crítica
- contador.c



- Desativar as interrupções ao entrar na seção crítica e reativar ao sair .

Por que é perigoso?

- Se a tarefa travar, o sistema inteiro trava.
- Pode causar perda de dados de rede/disco.
- **Não funciona em Multiprocessadores** (outros núcleos continuam rodando).

Uso restrito ao núcleo (Kernel) do SO.



- Usamos uma variável global que funciona como uma **flag** indicando se a seção crítica está:
 - Ocupada (`busy = 1`)
 - Livre (`busy = 0`).

```
int busy = 0 ; // a seção está inicialmente livre

void enter (){
    while (busy) ; // espera enquanto a seção estiver ocupada
    busy = 1 ; // marca a seção como ocupada
}

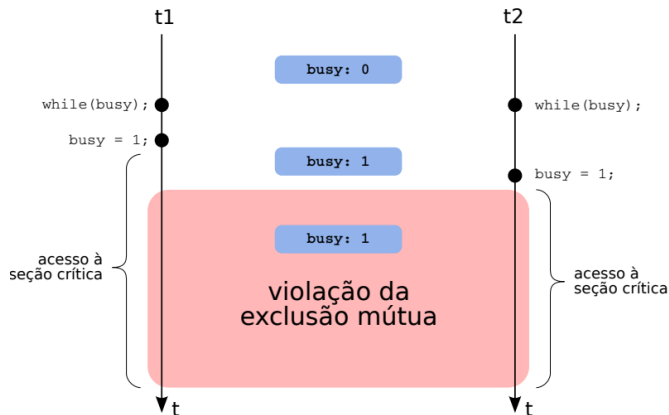
void leave (){
    busy = 0 ; // libera a seção (marca como livre)
}
```

- `enter()` fica em um loop de espera ativa (`busy waiting`) até que a seção esteja livre.
- Quando entra, marca como ocupada.
- `leave()` libera a seção crítica.

Vamos testar!



A solução trivial



O Defeito Fatal

O teste da variável e a atribuição ocorrem em momentos distintos. Uma troca de contexto entre eles permite que duas tarefas leiam "busy=0" e entrem juntas.

- **Espera Ocupada (Busy Wait):** O teste contínuo consome processador inutilmente.
 - a thread fica presa em um loop consumindo CPU enquanto espera, desperdiçando recursos.

Analogia - Banheiro de Bar

- Se duas pessoas chegarem ao mesmo tempo e olharem a plaquinha "livre", ambas podem tentar entrar juntas.
- Além disso, quem espera fica parado olhando a porta (espera Ocupada), em vez de poder fazer outra coisa até o banheiro ficar disponível.



- Existe uma variável global `turn` que indica qual tarefa tem o direito de entrar na seção crítica.
- Cada tarefa só entra quando `turn == task`.
- Ao sair, a tarefa atual passa o turno para a próxima $((turn + 1) \% num_tasks)$.

```
int num_tasks;
int turn = 0; // inicia pela tarefa 0

void enter(int task) // task vale 0, 1, ..., num_tasks-1
{
    while (turn != task) {
        // a tarefa espera seu turno
    }
}

void leave(int task)
{
    turn = (turn + 1) % num_tasks; // passa para a próxima tarefa
}
```



Analogia - Banheiro de Bar

- A variável turn é como uma senha numerada (0, 1, 2, ...).
- Cada pessoa só pode entrar quando sua senha for chamada.
- Ao sair, ela passa a vez para a próxima senha.

Ninguém entra fora de ordem e todos têm sua vez garantida.

Vamos testar!



Problema da Dependência

Se for a vez da Tarefa 0, mas ela não quiser entrar na seção crítica, a Tarefa 1 fica bloqueada para sempre esperando seu turno.

Analogia - Banheiro de Bar

- A Pessoa 1 pegou a senha, mas não está com vontade de usar o banheiro agora.
- A Pessoa 3 já bebeu bastante, está apertada e precisa urgentemente usar o banheiro.
- Porém, como o sistema é rígido, a Pessoa 3 só pode entrar quando chegar a vez da senha 3. Enquanto isso, ela precisa esperar a Pessoa 1, que nem tem necessidade de entrar.



Cada processo sinaliza que deseja entrar na seção crítica e cede a vez ao outro.

Se ambos quiserem entrar, a variável turn decide quem entra primeiro.

Progresso: se apenas um quer entrar, ele entra imediatamente.

Justiça: se ambos querem, o turn garante alternância, evitando que um monopolize o recurso.



Analogia - Banheiro de Bar

- Dois clientes (Pessoa A e Pessoa B)
- Cada pessoa coloca uma plaquinha “quero entrar” (isso é o flag).
- Além disso, existe uma plaquinha “vez” que aponta para A ou para B (isso é o turn).
- Se os dois querem entrar ao mesmo tempo:
 - A olha a plaquinha “vez”. Se estiver apontando para B, A espera.
 - B olha a plaquinha “vez”. Se estiver apontando para A, B espera.
- Assim, mesmo que os dois estejam apertados, apenas um entra de cada vez, e o outro espera sua vez.
- Quando alguém sai, tira sua plaquinha de “quero entrar” e o banheiro fica livre para o outro.



Algoritmo de Peterson (1981)

- A tarefa expressa o desejo de entrar, mas gentilmente cede o turno para a outra.
- Se a outra não quiser, ela entra imediatamente.

```
int turn = 0;           // indica de quem é a vez
int wants[2] = {0, 0}; // indica se a tarefa i quer acessar a seção crítica

void enter(int task) { // task pode valer 0 ou 1

    int other = 1 - task; // indica a outra tarefa
    wants[task] = 1;      // task quer acessar a seção crítica
    turn = other;
    while ((turn == other) && wants[other]) {
        // espera ocupada
    }
}

void leave(int task) {
    wants[task] = 0; // task libera a seção crítica
}
```

- Uma solução elegante para **duas tarefas** que combina a intenção (wants) e a vez (turn).
- Garante exclusão mútua e espera limitada.



- **Definição:** instruções que o processador executa de forma indivisível — ou acontecem por completo, ou não acontecem.
- **Benefício:** garantem consistência dos dados em ambientes concorrentes (múltiplas threads/processos).
- **Exemplo clássico:** incremento de variável compartilhada.
 - `saldo++` pode ser quebrado em: ler, somar, escrever.
 - Se não for atômico, outra thread pode interferir entre essas etapas.
- **Características:**
 - Usam instruções da CPU para garantir indivisibilidade.
 - São leves e rápidas.
 - Ideais para operações simples (incremento, troca de valor).
 - Para trechos maiores de código, ainda é necessário exclusão mútua.



Operações atômicas - TSL

- A solução definitiva para a variável busy: instruções de máquina que testam e alteram um valor em um único passo indivisível.
- Instruções de máquina específicas para exclusão:
 - **TSL (Test-and-Set Lock)**: Lê o valor antigo e escreve 1 atômicamente.
 - **XCHG (Exchange)**: Troca valores entre registrador e memória.

```
int lock = 0; // variável de trava

void enter(int *lock){ // passa o endereço da trava
    while (TSL(*lock)) {
        // espera ocupada sobre a trava
    }
}

void leave(int *lock) {
    *lock = 0; // libera a seção crítica
}
```

- Comportamento da instrução TSL

$$TSL(x) = \begin{cases} \text{old} \leftarrow x & // \text{guarda o valor anterior} \\ x \leftarrow 1 & // \text{ativa o flag} \\ \text{return}(\text{old}) & // \text{devolve o valor anterior} \end{cases}$$



Imagine que x é um cadeado ($0 =$ aberto, $1 =$ fechado).

- 1 `old <- x`: Você tira uma "foto" do estado atual do cadeado.
- 2 `x <- 1`: Sem soltar a maçaneta, você gira a chave para trancar (independente de já estar trancado ou não, você garante que agora está).
- 3 `return(old)`: Você olha para a "foto" que tirou no passo 1 para saber o que aconteceu.

O processo que chama o TSL interpreta o resultado da seguinte forma:

- Se o retorno for 0: "Opa, a foto mostrou que estava aberto! Como eu acabei de trancar (no passo 2), agora a sala é minha." → Entra na Seção Crítica.
- Se o retorno for 1: "Ih, a foto mostrou que já estava trancado. Alguém já está lá dentro." → Fica em loop (espera ocupada) tentando de novo.



- **Ineficiência:** uso de espera ocupada (teste contínuo de uma condição).
 - As tarefas ficam consumindo CPU sem necessidade.
 - O ideal seria suspender a tarefa até o recurso ficar disponível.
- **Injustiça:** não há garantia de ordem no acesso.
 - Uma tarefa pode entrar e sair várias vezes em sequência.
 - Outras tarefas podem ficar esperando indefinidamente.
- Essas soluções são usadas apenas:
 - Dentro do núcleo do Sistema Operacional.
 - Em sistemas muito simples ou didáticos.

Próximo passo: usar mecanismos mais robustos como *mutexes* e *semáforos*.



Atividade



```
int saldo = 100;
void* depositar(void* arg) {
    int valor = *(int*)arg;
    int temp = saldo;           // Passo 1: Leitura
    // Estou forçando a troca de contexto aqui:
    // A thread pausa por alguns microssegundos,
    // dando chance para a outra thread ler o mesmo saldo original.
    usleep(100);
    temp += valor;              // Passo 2: Soma
    saldo = temp;               // Passo 3: Escrita
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int v1 = 50, v2 = 30;
    pthread_create(&t1, NULL, depositar, &v1);
    pthread_create(&t2, NULL, depositar, &v2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Saldo final esperado: 180\n");
    printf("Saldo final obtido: %d\n", saldo);
    return 0;
}
```

- Aplicar os algoritmos abaixo no problema da conta
 - Alternância de Uso
 - Algoritmo de Peterson
 - TSL



Mecanismos de coordenação



■ No tópico anterior:

- Discutimos o problema das condições de disputa entre tarefas concorrentes.
- Vimos a necessidade de executar operações em **exclusão mútua** para evitar essas disputas.
- Observamos que as soluções apresentadas eram ineficientes e injustas na distribuição do acesso à seção crítica.

■ Neste tópico:

- Apresentamos mecanismos de sincronização mais sofisticados, como **semáforos** e **mutexes**.
- Esses mecanismos garantem eficiência e justiça no acesso à seção crítica.
- São amplamente utilizados na construção de aplicações concorrentes.



Semáforos



- Inventado por Dijkstra, o semáforo é um contador inteligente.
- Ele controla o acesso a um recurso que possui múltiplas instâncias ou um limite de capacidade.
- O semáforo provê:
 - **Eficiência:** baixo consumo de CPU
 - **Justiça:** respeita a ordem das requisições.
 - Independência de outras tarefas.
- Uso típico: controle de múltiplos acessos simultâneos (ex.: limitar número de threads que acessam um recurso)
- Exemplo Real: Limitar o número de conexões simultâneas a um banco de dados para não sobrecarregar o servidor.



Base de muitos mecanismos de comunicação e coordenação!



- O semaforo é um mecanismo de sincronização usado para controlar o acesso de múltiplas threads ou processos a recursos compartilhados, evitando conflitos e garantindo ordem.
- Um semáforo s é uma **variável composta** por:
 - Contador inteiro $s.counter$
 - Fila de tarefas $s.queue$ (inicia vazia)
 - Operações de acesso $down(s)$ e $up(s)$
- Imagine um “objeto semáforo” com atributos e métodos.

No fundo é uma ferramenta de sincronização que funciona como um contador de permissões.



É feito usando operações atômicas:

Operação wait(s) ou down(s) (ou P)

- **Decrementa** o contador (i) do semáforo.
- Se $i < 0$, **suspende** a tarefa e a põe na fila de s.
- Ou seja, se o valor for negativo, o processo é bloqueado.

Operação signal() ou up(s) (ou V)

- **Incrementa** o contador (i) do semáforo.
- Se $i \leq 0$, **acorda** uma tarefa da fila - devolve a permissão.
- Ou seja, se houver processos bloqueados, libera um deles.



Restaurante com 10 mesas - Semáforo como contador

- Cada cliente que chega "decrementa"o contador de mesas.
- Se o contador chegar a 0, os próximos clientes esperam na fila (ficam bloqueados).
- Quando alguém termina de comer e sai, "incrementa"o contador e o próximo da fila é chamado.

Bar tem apenas um banheiro - Semáforo binário

A chave fica pendurada em um prego atrás do balcão.

- Estado Inicial (Valor = 1): A chave está no prego. O banheiro está livre.
- Operação down: Um cliente chega, pega a chave e vai ao banheiro. O prego agora está vazio (Valor = 0). Se outro cliente chegar e olhar para o prego, ele vê que não há chave e senta para esperar.
- Operação up: O cliente sai do banheiro e devolve a chave para o prego. O valor volta a ser 1. Nesse momento, o próximo cliente da fila pode pegar a chave.



```
1: procedure DOWN( $t, s$ )  
2:    $s.counter \leftarrow s.counter - 1$   
3:   if  $s.counter < 0$  then  
4:      $\text{append}(t, s.queue)$   
5:      $\text{suspend}(t)$   
6:   end if  
7: end procedure
```

- ▷ põe t no final de $s.queue$
- ▷ a tarefa t perde o processador

```
8: procedure UP( $s$ )  
9:    $s.counter \leftarrow s.counter + 1$   
10:  if  $s.counter \leq 0$  then  
11:     $u = \text{first}(s.queue)$   
12:     $\text{awake}(u)$   
13:  end if  
14: end procedure
```

- ▷ retira a primeira tarefa de $s.queue$
- ▷ devolve u à fila de tarefas prontas

```
15: procedure INIT( $s, v$ )  
16:    $s.counter \leftarrow v$   
17:    $s.queue \leftarrow [ ]$   
18: end procedure
```

- ▷ valor inicial do contador
- ▷ a fila inicia vazia



- Cada recurso é representado por um semáforo:
 - Operações down(s) para obter e up(s) para liberar.
 - Iniciar o semáforo com o contador em 1.
 - Somente uma tarefa obtém o semáforo por vez.

```
init (s, 1) ; // semáforo que representa a conta

void depositar (semaphore s, int *saldo, int valor)
{
    down (s) ; // solicita acesso à conta
    (*saldo) += valor ; // seção crítica
    up (s) ; // libera o acesso à conta
}
```

- Diferença dos mecanismos que vimos antes: No TSL, a thread fica na porta batendo sem parar (gasta energia).
- No Semáforo, a thread senta e dorme; o SO a acorda quando a chave volta pro gancho.



Algumas chamadas da API POSIX para semáforos:

```
include <semaphore.h>

// inicializa um semáforo, com valor inicial "value"
int sem_init (sem_t *sem, int pshared, unsigned int value);

// Up(s)
int sem_post (sem_t *sem);

// Down(s)
int sem_wait (sem_t *sem);
```



Exemplo 1 - Contador

- **Cenário:** 100 threads incrementando 100.000 vezes a mesma variável `sum`.
- **O Problema:** Sem sincronização, o resultado seria inferior a 10.000.000 devido a atualizações perdidas.
- **A Solução:** `sem_wait(&s)` e `sem_post(&s)` garantem acesso exclusivo, isto é, evitar condição de corrida.

Compilação

```
gcc -Wall m5-semaforo.c -o m5-semaforo -lpthread
```

O que observar?

- **Exclusão Mútua:** O valor inicial do semáforo é 1. Apenas uma thread "passa" pelo wait por vez.
- **Desempenho:** Note como a execução é ligeiramente mais lenta que a versão sem proteção, devido ao *overhead* de sincronização do Sistema Operacional.



Exemplo 2 - Estacionamento

- Estacionamento com acesso por cancelas de entrada e de saída



- Imagine um estacionamento com 10 vagas.
- O semáforo começa com valor 10.
- Cada carro que entra faz wait() (decrementa).
- Quando o valor chega a 0, não há mais vagas e os próximos carros precisam esperar.
- Quando um carro sai, faz signal() (incrementa).

```
// inicia o semáforo (10 vagas)
init (vagas, 10) ;

// cancela de entrada, p/ cada carro
void obtem_vaga ()
{
    // solicita uma vaga
    down (vagas) ;
}

// cancela de saída, p/ cada carro
void libera_vaga ()
{
    // libera uma vaga
    up (vagas) ;
}
```



■ gcc estacionamento.c -o estacionamento -lpthread

- O conceito principal é tratar o número de vagas como um recurso compartilhado
- O semaforo é um contador atômico
 - `sem_wait()`: Tenta decrementar o contador. Se for 0 (estacionamento lotado), a thread (carro) bloqueia e espera.
 - `sem_post()`: Incrementa o contador, sinalizando que uma vaga foi liberada e permitindo que o próximo carro da fila entre.

Mecânica do Semáforo (`sem_t`)

- `sem_init(&vagas, 0, 5)`: Inicializa o contador de recursos disponíveis.
- `sem_wait()`: Operação P (*proberen*). Decrementa ou bloqueia se for 0.
- `sem_post()`: Operação V (*verhogen*). Incrementa e acorda threads.

Comportamento do Sistema

- **Bloqueio Automático**: O 6º carro entra em estado *Wait* sem consumir CPU.
- **Gerenciamento de Quantidade**: Ideal para pools de recursos, diferente do Mutex (binário).



Exemplo 3 - Limitação de Acesso ao Banco de Dados

- Vamos simular um servidor com 10 threads tentando realizar operações, mas com um limite físico de apenas **3 conexões simultâneas** ao Banco de Dados.

Configuração do Sistema

- **MAX_CONEXOES** 3: Define o teto de recursos disponíveis.
- **sem_init(&bd, 0, 3)**: O semáforo nasce com valor 3.
- **Recurso Compartilhado**: O Banco de Dados é o recurso crítico.

Funcionamento

- **Admissão**: As primeiras 3 threads passam pelo wait imediatamente.
- **Bloqueio**: As outras 7 threads ficam em fila (estado *Waiting*).
- **Liberação**: O post sinaliza que uma "vaga" no pool abriu.

Compilação

```
gcc -Wall m6-semaforo.c -o m6 -lpthread
```

IMPORTANTE

Temos o conceito de **estrangulamento**: controlamos a vazão do sistema para proteger a integridade do recurso mais lento (o Banco de Dados).



Exemplo 4 - Produtor-Consumidor com Semáforos

- Vamos simular o gerenciamento de um buffer compartilhado de tamanho limitado ($N = 5$), onde o Produtor não pode "estourar" o espaço e o Consumidor não pode ler o vazio.
 - Se estiver cheio, o produtor precisa esperar.
 - Se estiver vazio, o consumidor precisa esperar.

Três Semáforos em Ação

- `sem_vagas` (5): Contador de espaços disponíveis. Controla o bloqueio do produtor.
- `sem_itens` (0): Contador de itens prontos. Controla o bloqueio do consumidor.
- `semaforo` (1): **Trava de segurança.** Garante que apenas uma thread manipule o buffer por vez.

A Ordem dos waits

- **Verificação:** Primeiro checka-se se pode agir (`vagas/itens`).
- **Acesso:** Depois, pega-se a "chave" (semaforo) para alterar os dados.
- **Circularidade:** Uso do operador % para reaproveitar as posições do vetor.

Compilação

```
gcc -Wall m7-semaforo.c -o m7 -lpthread
```

Conceito de Sincronização

Diferente do *Pool de Conexões*, aqui existe uma **dependência cruzada**: o Produtor "acorda" o Consumidor (`post(itens)`) e o Consumidor "acorda" o Produtor (`post(vagas)`).



1 Pool de Conexões de Banco de Dados

- Um servidor web não pode abrir conexões infinitas com o banco.
- Criamos um semáforo com valor, por exemplo, 10.
- Mesmo com 1000 usuários, apenas 10 threads acessam o banco simultaneamente.
- As demais aguardam sua vez de forma organizada.

2 Controle de Buffer (Produtor/Consumidor)

- Um semáforo controla as vagas vazias (evita transbordo do buffer).
- Outro controla os itens prontos (evita consumo de itens inexistentes).

3 Limitação de Download (Throttling)

- Aplicativos como Steam ou Torrent limitam downloads simultâneos.
- Exemplo: semáforo inicializado em 3 → apenas 3 downloads ativos.
- Ao terminar um arquivo, libera uma vaga para o próximo.



■ Múltiplos Recursos

- Use quando há **mais de uma instância** de um recurso.
- Exemplo: pool de conexões de banco de dados.

■ Sincronização de Eventos

- Use quando uma thread precisa **notificar outra**.
- Exemplo: produtor sinalizando que há item disponível para o consumidor.

■ Gerenciamento de Filas e Pools

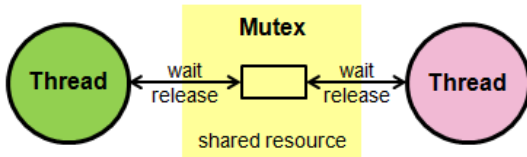
- Use para controlar acesso organizado e evitar sobrecarga.
- Exemplo: limitar número de downloads simultâneos.



Mutexes



- É um mecanismo de sincronização simplificado que possui apenas dois estados: **travado (locked)** ou **destravado (unlocked)**.
- É um semáforo binário usado para garantir exclusão mútua.
- É a ferramenta ideal quando o objetivo é apenas proteger uma Seção Crítica.
 - Em um ambiente com várias threads, apenas uma consiga acessar um determinado recurso (variável, arquivo, tabela) por vez.



- Uso: Proteger regiões críticas (trechos de código que acessam variáveis compartilhadas).
- Exemplo Real: O acesso a uma impressora. Apenas um documento pode ser impresso por vez para não misturar as páginas.

Pense nele como um "porteiro" que garante que ninguém se atropele.



Banheiro do bar

- A Regra: Só cabe uma pessoa por vez.
- O Mutex (Cadeado): Se a porta está verde, você entra e tranca a porta por dentro.
- O Bloqueio: Enquanto você está lá dentro, outras pessoas chegam, veem que está ocupado e formam uma fila (ficam em estado de espera). Elas não tentam derrubar a porta; elas simplesmente param e esperam.
- A Liberação: Só quando você sai e destranca a porta é que a próxima pessoa da fila pode entrar.

O Microfone da Palestra

Em uma rodada de perguntas onde há apenas um microfone:

- Quem segura o microfone tem o "direito de falar" (acesso ao recurso).
- Se outra pessoa quer falar, ela precisa esperar o microfone ser entregue.
- Diferença crucial: No Mutex, você não pode "roubar" o microfone. O dono atual precisa voluntariamente soltá-lo.



1 lock() — Trancar

- A thread tenta pegar a chave.
- Se já estiver em uso, a thread "dorme" até ser liberada.

2 Seção Crítica

- Trecho de código que manipula o recurso sensível.
- Deve ser o mais curto possível para evitar travamentos.

3 unlock() — Destrancar

- A thread devolve a chave.
- O Sistema Operacional acorda a próxima thread da fila.

4 Regra de Ouro — Diferença para Semáforo

- No **Mutex**, existe o conceito de **proprietário**.
- Quem trancou é o único que pode destrancar.
- Se a Thread A deu lock, a Thread B não pode chegar e dar unlock à força.
- Isso garante hierarquia de segurança que o semáforo binário não possui por padrão.

Conceito Importante

Diferente do Semáforo, o Mutex não serve para "contar vagas". Ele é o "cadeado" que garante que a operação `sum += 1` seja **atômica**.



■ Semáforo simplificado: livre ou ocupado

```
#include <pthread.h>

// inicializa mutex, usando um struct de atributos
int pthread_mutex_init (pthread_mutex_t *restrict mutex,
const pthread_mutexattr_t *restrict attr);

// destrói uma variável do tipo mutex
int pthread_mutex_destroy (pthread_mutex_t *mutex);

// solicita acesso à seção crítica protegida pelo mutex;
// se a seção estiver ocupada, bloqueia a tarefa
int pthread_mutex_lock (pthread_mutex_t *mutex);

// libera o acesso à seção crítica protegida pelo mutex
int pthread_mutex_unlock (pthread_mutex_t *mutex);
```



Exemplo 1: Contador

Estrutura do Código (m8-mutex.c)

- **Inicialização:** `pthread_mutex_init(&lock, NULL)` prepara o cadeado antes da criação das threads.
- **Seção Crítica:** O trecho entre `lock()` e `unlock()` torna-se **atômico**. Nenhuma thread consegue ler o valor de `sum` enquanto outra está somando.
- **Finalização:** `pthread_mutex_destroy(&lock)` libera os recursos do sistema operacional.

Compilação

```
gcc -Wall m8-mutex.c -o m8 -lpthread
```

Análise de Resultado

O Mutex garante que, enquanto a Thread A estiver fazendo os três passos (leitura, soma e escrita), ninguém mais consiga sequer ler o valor de `sum`.



Exemplo 2 - Sincronização de Início

- Threads "trabalhadoras" esperam um sinal do "chefe" para começarem a agir

A Mecânica do `pthread_cond_wait`

- 1 **Dorme e Libera:** A thread libera o Mutex e entra em estado de espera.
- 2 **Acorda e Tranca:** Ao receber o sinal, ela tenta re-adquirir o Mutex antes de continuar.
- 3 **O loop while:** Essencial para re-testar a condição e evitar "acordos espúrios" (quando a thread acorda sem um sinal real).

Sinalização

- `cond_signal`: Acorda apenas uma thread da fila.
- `cond_broadcast`: Acorda **todas** as threads (usado no código).

Compilação

- `gcc -Wall m9-mutex.c -o m9 -lpthread`



Exemplo 3 - Impressora

Cenário: Recurso Não Compartilhável

Duas threads querem usar a **mesma saída de texto**. Sem proteção, as mensagens se misturariam linha a linha.

Por que o Mutex resolve?

- **lock**: Quando o PC "A" começa a imprimir, ele tranca a porta.
- **Espera**: O PC "B" chega, vê que está trancado e fica em estado de espera (não gasta CPU).
- **unlock**: Só quando "A" termina, "B" recebe a chave do Sistema Operacional.

Compilação

```
gcc -Wall impressora.c -o impressora -lpthread
```

O que observar na execução

Mesmo que as threads rodem "ao mesmo tempo", o terminal mostrará o bloco do Computador A inteiro e **depois** o bloco do Computador B inteiro. Nunca misturados.



■ Atualização de Saldo Bancário

- Sem Mutex: duas threads leem o saldo (500), calculam 600 e gravam 600 → **Race Condition**.
- Com Mutex: primeira thread tranca, lê 500, grava 600 e destranca. Segunda thread lê 600 e grava 700 corretamente.

■ Escrita em Arquivos de Log

- Sem Mutex: várias threads escrevem ao mesmo tempo → texto corrompido (ex.: "Hellowo rld").
- Com Mutex: cada thread escreve sua mensagem completa sem interferência.

■ Acesso a Hardware (Impressoras/Scanners)

- Sem Mutex: relatórios de diferentes usuários podem sair intercalados.
- Com Mutex: garante que um relatório seja impresso inteiro antes do próximo.



■ Quando usar

- **Mutex**: proteger variáveis globais ou operações não atômicas.
- **Semáforo**: controlar acesso a recursos limitados (ex.: conexões de banco, vagas).

■ Diferença Crucial

- **Mutex**: tem um "dono"— quem tranca deve destrancar.
- **Semáforo**: não tem dono — qualquer thread pode liberar uma vaga.

■ Analogia

- **Mutex (Cadeado)**: só quem trancou pode abrir.
- **Semáforo Binário (Chave no Pregão)**: qualquer thread pode devolver a chave.



■ Use Mutex quando:

- Você tem uma variável global que duas threads querem alterar.
- Precisa garantir que uma não sobrescreva a outra.
- É puramente sobre **proteção de dados**.

■ Use Semáforo quando:

- Há múltiplos recursos compartilhados (ex.: 3 impressoras para 10 threads).
- Uma thread precisa esperar outra terminar antes de prosseguir (sincronização/sinalização).

■ Resumo:

- **Mutex**: é um **Cadeado** — serve para trancar.
- **Semáforo**: é um **Placar Eletrônico** — serve para contar e sinalizar.



Variável de condição



Cenário: Recurso Não Compartilhável

- Representa uma condição aguardada por uma tarefa.
 - Uma tarefa espera a condição ficar verdadeira.
 - Outra tarefa sinaliza que a condição é verdadeira.
-
- Ela permite que uma thread suspenda sua execução até que uma condição específica seja verdadeira.
 - Exemplo Real: Um Buffer de Produtor/Consumidor. O consumidor deve esperar (Variável de Condição) até que o buffer não esteja mais vazio.



Componentes da variável de condição c:

- Enquanto o Mutex serve para "trancar a porta", a Variável de Condição serve para "esperar por um evento específico" sem gastar processamento.
- Ela nunca trabalha sozinha: ela sempre precisa de um Mutex para funcionar.

Componentes

- Semáforo binário c.mutex
- Fila de tarefas c.queue
- Operações atômicas: wait(c), signal(c), broadcast(c)



■ Wait (Esperar)

- A thread diz: "Ainda não aconteceu o que preciso, vou dormir".
- Ao executar wait, o **mutex é liberado automaticamente**.
- Quando acorda, a thread recupera o mutex antes de continuar.

■ Signal (Sinalizar)

- Acorda apenas **uma thread** que esteja esperando.
- É como dar um "cutucão" no primeiro da fila.

■ Broadcast (Avisar todos)

- Acorda **todas as threads** que estão esperando pela condição.
- Útil quando uma mudança beneficia vários interessados ao mesmo tempo.



■ **Mutex:**

- É a porta de entrada do restaurante.
- Apenas um cliente fala com a recepcionista por vez.

■ **Wait:**

- Cliente recebe um beeper e vai esperar no sofá.
- Libera o Mutex para que outros possam falar com a recepcionista.

■ **Signal:**

- Quando a mesa fica livre, a recepcionista aperta o botão.
- O beeper de um cliente toca e ele volta para a porta (Mutex).

■ **Broadcast:**

- Se o restaurante inteiro estava fechado (ex.: falta de luz).
- Ao reabrir, todos os beepers tocam e todos os clientes podem tentar entrar.



t: tarefa que invocou a operação

c: variável de condição

m: *mutex* associado à condição

procedure WAIT(*t*, *c*, *m*)

append (*t*, *c.queue*)

unlock (*m*)

suspend (*t*)

lock (*m*)

end procedure

▷ põe *t* no final de *c.queue*

▷ libera o *mutex*

▷ a tarefa *t* é suspensa

▷ ao acordar, requer o *mutex*

procedure SIGNAL(*c*)

u = first (*c.queue*)

awake(*u*)

end procedure

▷ retira a primeira tarefa de *c.queue*

▷ devolve *u* à fila de tarefas prontas



Exemplo: produção e consumo de dados

```
int buffer[...] ; // buffer de dados
condition c ; // indica presença de dados no buffer
mutex m ; // mutex associado à condição c
```

- Obtém dados de alguma fonte (arquivo, etc)
- Deposita os dados em um buffer compartilhado
- Sinaliza que o buffer contém dados

```
task produce_data (){
    while (1){
        // obtem dados de alguma fonte (rede, disco, etc)
        retrieve_data (...) ;

        // insere dados no buffer
        lock (m) ; // acesso exclusivo ao buffer
        put_data (buffer, ...) ; // poe dados no buffer
        signal (c) ; // sinaliza c (buffer tem dados)
        unlock (m) ; // libera o buffer
    }
}
```

- Condição: há dados no buffer compartilhado



Exemplo: produção e consumo de dados

- Aguarda a presença de dados no buffer
- Retira os dados do buffer
- Usa os dados obtidos

```
task consume_data () {  
  while (1) {  
    // aguarda presença de dados no buffer  
    lock (m) ; // acesso exclusivo ao buffer  
    while (size (buffer) == 0) // enquanto buffer estiver vazio  
      wait (c, m) ; // aguarda sinal da condição c  
  
    get_data (buffer, ...) ; // retira os dados do buffer  
    unlock (m) ; // libera o buffer  
  
    process_data (...) ; // trata os dados recebidos  
  }  
}
```



■ `pthread_cond_wait(cond, mutex)`

- A thread entra em espera até que a condição seja sinalizada.
- Enquanto espera, o **mutex é liberado automaticamente**.
- Ao acordar, a thread recupera o mutex antes de continuar.

■ `pthread_cond_signal(cond)`

- Acorda **uma única thread** que esteja esperando na condição.
- Usado para notificar que houve uma mudança relevante.

■ `pthread_cond_broadcast(cond)`

- Acorda **todas as threads** que estão esperando na condição.
- Útil quando uma alteração beneficia múltiplos interessados ao mesmo tempo.



Exemplo 1 - Produção e Consumo de Dados

As Duas Tarefas:

- `produce_data`: Obtém dados, deposita no buffer compartilhado e **sinaliza** (`cond_signal`) que a condição mudou.
- `consume_data`: Tenta retirar dados. Se o buffer estiver vazio, entra em espera passiva (`cond_wait`) até ser acordada.

Compilação

```
gcc -Wall m10-vc.c -o m10 -lpthread
```

Destaque do Fluxo

- 1 O Consumidor tranca o Mutex e checa: `buffer == 0`?
- 2 Se sim, chama `pthread_cond_wait` (solta o Mutex e dorme).
- 3 O Produtor ganha o Mutex, coloca o dado e chama `cond_signal`.
- 4 O Consumidor acorda, re-adquire o Mutex e processa o dado.



Exemplo 2 - Posto de Gasolina

- Um frentista (Thread A) não deve ficar gastando energia olhando para a rua se não há carros.
- Ele deve dormir e ser acordado apenas quando um cliente (Thread B) chegar.
- **A Condição:** `cliente_chegou == 1`.
- **O Problema:** O frentista precisa de uma forma de "ser acordado" sem ficar perguntando "já chegou?" 1 milhão de vezes por segundo.

Compilação

```
gcc -Wall m11-vc.c -o m11 -lpthread
```

Papéis na Sincronização

- **Mutex:** É a "garantia de exclusividade" no atendimento.
- **Wait:** É o frentista fechando os olhos e liberando o acesso ao posto.
- **Signal:** É a buzina do cliente que "acorda" quem está no wait.



■ Buffers (Produtor/Consumidor)

- Consumidor dá `wait` se o buffer estiver vazio.
- Produtor dá `wait` se o buffer estiver cheio.
- Ambos acordam quando recebem `signal` do outro.

■ Leitores e Escritores

- Vários leitores podem acessar simultaneamente.
- Escritor deve esperar até que todos os leitores terminem.
- Escritor acorda quando o último leitor sinaliza.

■ Barreiras de Sincronização

- Exemplo: jogo online que só começa com 10 jogadores.
- Os primeiros 9 dão `wait`.
- O 10º jogador dá `broadcast`, acordando todos para iniciar.



Por que não usar apenas Semáforo?

■ Semáforo:

- É apenas um contador simples.
- Não permite expressar condições complexas.

■ Variável de Condição:

- Permite lógica mais sofisticada.
- Exemplo: `while (saldo <= 100 || !cartao_ativo) wait();`
- Só prossegue quando a condição desejada é verdadeira.



Monitores



- O Monitor é um conceito de mais alto nível (muito comum em Java).
- É um "pacote" que encapsula os dados, as funções e a sincronização (Mutex + Condição) em um único lugar.
- Simplifica a programação concorrente, pois a linguagem garante exclusão mútua automaticamente.

Definição

- Estrutura de sincronização associada a um recurso
- Requer e libera a seção crítica de forma transparente.
- O programador não precisa mais se preocupar com isso.



- Um Monitor é um "pacote" que combina três coisas em um só lugar:
 - Dados Privados: As variáveis que precisam de proteção.
 - Exclusão Mútua Automática: Um Mutex interno que garante que apenas uma thread execute os métodos do objeto por vez.
 - Variáveis de Condição: Mecanismos (wait e notify) para as threads esperarem por eventos.

A grande vantagem

Você não precisa dar lock e unlock manualmente toda hora. Ao entrar em um método "sincronizado", o Monitor tranca a porta para você automaticamente.



■ **Mutex — A Porta de Entrada**

- Apenas um cliente fala com o gerente por vez.
- Se ocupado, os demais esperam na fila externa.

■ **Variável de Condição — Sala de Espera Interna**

- Cliente pede saque, mas o dinheiro não chegou.
- Ele recebe um beeper e vai esperar, liberando o Mutex.

■ **Wait**

- Cliente dorme na sala de espera.
- O gerente pode atender outros clientes enquanto isso.

■ **Notify (Signal)**

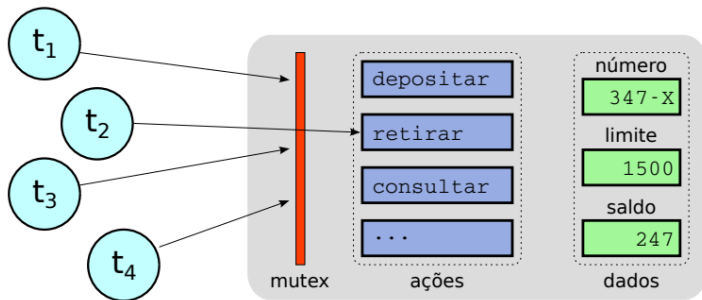
- Outro cliente deposita dinheiro.
- Esse evento acorda o cliente que estava esperando.

■ **Saída**

- Cliente acordado volta ao gerente.
- Faz o saque e libera a mesa para o próximo.



- Pense em um monitor como um “objeto sincronizado”.



- Garante que apenas uma das ações (depositar ou retirar) podem estar ativa em um instante
 - Não pode ter 2 operações concorrentes de retirar ou 2 de depositar ou 1 de depositar concorrendo com 1 de retirar.



```
monitor conta {
    float saldo = 0 ; // recurso (variáveis)
    float limite ;

    void depositar (float valor) { // ação sobre o recurso
        if (valor >= 0)
            conta->saldo += valor ;
        else
            error ("erro: valor negativo\n") ;
    }

    void retirar (float saldo) { // ação sobre o recurso
        if (valor >= 0)
            conta->saldo -= valor ;
        else
            error ("erro: valor negativo\n") ;
    }
}
```



```
class Conta {  
    private float saldo = 0;  
  
    public synchronized void depositar (float valor) {  
        if (valor >= 0)  
            saldo += valor ;  
        else  
            System.err.println("valor negativo");  
    }  
  
    public synchronized void retirar (float valor){  
        if (valor >= 0)  
            saldo -= valor ;  
        else  
            System.err.println("valor negativo");  
    }  
}
```



1. ContadorSemMonitor

- **Falha:** Múltiplas threads acessam o atributo valor simultaneamente.
- **Intercalação:** Operação ++ é interrompida entre a leitura e a escrita.
- **Efeito:** Saldo final inconsistente e abaixo do esperado.

2. ContadorComMonitor

- **Solução:** Uso da palavra-chave `synchronized` no método.
- **Controle:** A JVM garante que apenas uma thread use o Monitor do objeto por vez.
- **Efeito:** Execução serializada e resultado final 100% íntegro.

Conceito Chave: Exclusão Mútua em Java

- **Atomicidade:** O Monitor garante que o incremento (*read-modify-write*) não seja interrompido por outras threads concorrentes.
- **Monitor Intrínseco:** Em Java, cada objeto possui seu próprio cadeado (*lock*). O `synchronized` é a porta de entrada para esse mecanismo.



Exemplo 1 - Conta Bancária

Comparação entre o acesso desprotegido e o acesso via Monitor, simulando múltiplos depósitos simultâneos com atraso de contexto (*sleep*).

1. ContaSemMonitor

- **Falha:** Várias threads leem o mesmo saldo inicial.
- **Resultado:** Atualizações perdidas (*Lost Updates*).
- **Efeito:** Saldo final inconsistente (ex: 100 em vez de 1000).

2. ContaComMonitor

- **Solução:** Uso do `synchronized` no método `depositar()`.
- **Bloqueio:** Thread segura o *lock* mesmo durante o *sleep*.
- **Efeito:** Execução serializada e saldo 100% íntegro.

Conceito Chave: Exclusão Mútua

- **Atomicidade:** O Monitor transforma um conjunto de passos (ler, somar, gravar) em uma operação única do ponto de vista externo.
- **Visibilidade:** Garante que a próxima thread veja a alteração feita pela anterior.



Exemplo 2 - A Pizzaria (Produtor-Consumidor)

Coordenação entre duas threads através de um objeto monitor que gerencia o acesso a um recurso finito (a Mesa).

1. Classe Mesa (O Monitor)

- **synchronized:** Garante que só um método seja executado por vez.
- **wait/notify:** Gerencia a "sala de espera" das threads.

2. Threads (Cozinheiro/Entregador)

- **Cozinheiro:** Tenta colocarPizza(). Se a mesa estiver cheia, ele dorme.
- **Entregador:** Tenta retirarPizza(). Se estiver vazia, ele dorme.
- **Sinalização:** Um acorda o outro ao mudar o estado.

O que observar?

- **Exclusão Mútua:** O Monitor impede que o Entregador tente retirar algo enquanto o Cozinheiro ainda está colocando.
- **Sincronização de Condição:** Diferente da memória compartilhada, aqui o Leitor (Entregador) espera **bloqueado** até que o dado exista.
- **Lógica de Negócio:** O controle de "mesa cheia" ou "vazia" fica interno ao objeto, simplificando o uso pelas threads.



O Cenário: O Banheiro Coletivo (Recurso $K = 3$)

Múltiplas threads (**Pessoas**) disputam um número limitado de instâncias de um recurso (**Cabines**), gerenciadas pela classe Banheiro.

Componentes do Exemplo

- **Classe Banheiro:** Atua como o **Monitor**, encapsulando a variável `cabinesLivres`.
- **Classe MainBanheiro:** Orquestra a criação de N threads para gerar a disputa.

Mecanismos de Controle

- `entrar()`: Bloqueia a thread via `wait()` se o pool estiver vazio ($K = 0$).
- `sair()`: Incrementa o pool e sinaliza a liberação via `notifyAll()`.

O que observar

- **Controle de Admissão:** O Monitor impede que o sistema entre em estado inválido (recursos negativos).
- **Sincronização Indireta:** As threads não se comunicam entre si; elas interagem apenas com o estado do Monitor.
- **Abstração de Hardware:** O programador foca na lógica (*vagas disponíveis*), enquanto a JVM cuida da fila de prontos.



Resumindo



1 Mutex (Cadeado de Posse)

- Exclusão mútua simples: apenas uma thread acessa o recurso por vez.
- Preciso proteger esta variável para que apenas uma thread altere o valor por vez.

2 Semáforo (Contador de Vagas)

- Contador atômico para controlar recursos limitados ou sinalização.
- Tenho um limite de 5 conexões simultâneas e preciso gerenciar a fila.

3 Variável de Condição (Alarme de Evento)

- Permite que uma thread durma até que uma condição específica mude.
- Quero que esta thread só acorde quando houver um novo item na lista.

4 Monitor (Estrutura Organizada)

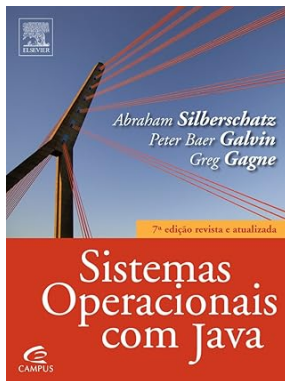
- Abstração que encapsula dados, Mutex e Variáveis de Condição em um objeto.
- Quero que os métodos da minha classe se protejam automaticamente.



- Proteção simples de variável? Mutex.
- Gerenciar um pool de recursos (várias vagas)? Semáforo.
- Thread precisa esperar um evento específico? Variável de Condição.
- Está programando em Java/C e quer código limpo? Monitores (synchronized).



Exemplos retirados de



- SILBERSCHATZ, Abraham. Sistemas operacionais com Java. Elsevier Brasil, 2008.
- MAZIERO, C. Sistemas Operacionais: Conceitos e Mecanismos. Editora da UFPR, 2019. 456 p. ISBN 978-85-7335-340-2.



Atividade



Disponível no sigaa

